2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)

2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C) | 979-8-3315-7990-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/MODELS-C68889.2025.00019

AI-assisted JSON Schema Creation and Mapping

1st Felix Neubauer, 3rd Benjamin Uekermann
Institute for Parallel and Distributed Systems
University of Stuttgart
Stuttgart, Germany
Felix.Neubauer@ipvs.uni-stuttgart.de,
Benjamin.Uekermann@ipvs.uni-stuttgart.de

2nd Jürgen Pleiss
Institute of Biochemistry and Technical Biochemistry
University of Stuttgart
Stuttgart, Germany
Juergen.Pleiss@itb.uni-stuttgart.de

***Abstract*—Model-Driven Engineering (MDE) places models at the core of system and data engineering processes. In the context of research data, these models are typically expressed as schemas that define the structure and semantics of datasets. However, many domains still lack standardized models, and creating them remains a significant barrier, especially for non-experts. We present a hybrid approach that combines large language models (LLMs) with deterministic techniques to enable JSON Schema creation, modification, and schema mapping based on natural language inputs by the user. These capabilities are integrated into the open-source tool *MetaConfigurator*, which already provides visual model editing, validation, code generation, and form generation from models. For data integration, we generate schema mappings from heterogeneous JSON, CSV, XML, and YAML data using LLMs, while ensuring scalability and reliability through deterministic execution of generated mapping rules. The applicability of our work is demonstrated in an application example in the field of chemistry. By combining natural language interaction with deterministic safeguards, this work significantly lowers the barrier to structured data modeling and data integration for non-experts.**

***Index Terms*—rdm, research data management, mde, model driven engineering, json, yaml, configuration, schema, data, model, editor, gui, tool, ai, llm, mapping, matching**

I. Introduction

Model-Driven Engineering (MDE) emphasizes the central role of models in the design, development, and operation of systems [1]. In the context of research data management [2]–[4], models are typically expressed as schemas, which define the structure and semantics of datasets. These models enable key functionalities such as instance validation, transformation, code generation, and visualization, and are essential for ensuring data quality, interoperability, and reusability.

Despite their importance, many scientific domains still lack standardized data models. In fields such as chemistry, data are often stored in electronic lab notebooks (ELNs) or spreadsheets, where much of the meaning is implicit rather than explicitly structured. While CSV documents are used in many machine learning workflows, they are inherently limited to flat, table-like structures and cannot encode relationships, constraints, or domain-specific rules. This lack of structure restricts interoperability, reproducibility, and the application of downstream methods such as knowledge graph construction or automated schema-driven validation.

Recent advances in large language models (LLMs) offer a promising opportunity to bridge this gap by assisting users, particularly non-experts, in the creation and refinement of data models. LLMs can translate natural language descriptions into structured model representations, potentially democratizing model-driven techniques. However, relying solely on LLMs introduces several limitations: 1) lack of guaranteed model validity, 2) limited interpretability of plain-text outputs, 3) challenges in processing large or complex datasets, and 4) the need for users to craft effective prompts.

To address these issues, we present an MDE-based approach that integrates LLMs with deterministic safeguards, including targeted pre- and post-processing, and the rule-based execution of AI-generated mappings, implemented in the open-source tool *MetaConfigurator*[1] [5]. MetaConfigurator supports interactive visual model (schema) editing, validation, and code/-documentation/form generation.

We extend the tool with capabilities for schema creation, modification, and querying from natural language input (Section III-A). By integrating prompt engineering, context management, and post-processing safeguards with visual feedback and schema validation, we align LLM-generated content with the principles of model correctness and transparency central to MDE. Second, we introduce a model-driven approach to data integration (Section III-B): given heterogeneous data sources in JSON [6], YAML [7], XML [8], or CSV, LLMs generate human-readable mapping rules to transform the data into a target schema (model). These rules are executed deterministically, separating generation from execution and ensuring reliability and scalability, especially for large datasets. To demonstrate practical applicability, Section IV showcases an example from the domain of chemistry, where existing unstructured Excel data are transformed into structured, interoperable, and AI-ready representations using the extended toolchain.

II. Background and Related Work

This section introduces JSON, JSON Schema and work related to schema creation, schema mapping, large language models (LLMs) and prompt engineering.

Deutsche Forschungsgemeinschaft (DFG) under project numbers 528693298 (preECO), 358283783 (SFB1333), and 390740016 (EXC2075)

[1]https://github.com/MetaConfigurator/meta-configurator

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on March 10,2026 at 14:36:17 UTC from IEEE Xplore. Restrictions apply.

A. JSON and JSON Schema

JSON is a widespread data format for exchanging information with web services, storing semi-structured documents in NoSQL databases [9], and representing structured content in APIs and configuration files. Its simplicity and human-readability, combined with wide support across programming languages, make it a natural fit for data interchange and machine learning pipelines.

JSON Schema, the de-facto standard for describing the structure and constraints of JSON documents [10], plays a crucial role in enabling validation, interoperability, and automation. It can also be understood as a modeling language, capturing the conceptual structure of data much like class diagrams in MDE.

B. JSON Schema Creation

There exist several so-called JSON schema editors, which are tools for creating and editing schemas. Among others, this includes MetaConfigurator [5], Adamant [11] and Liquid Studio JSON Schema Editor[2] (paid). All these editors require some level of understanding of JSON schema. For this work, we build on MetaConfigurator a general-purpose schema editor and form generator that supports different data formats (e.g., JSON, YAML, XML) and different ways to present and edit the data (e.g., a text editor, GUI editor or schema diagram [12]). We choose MetaConfigurator, because of its modular architecture and because it can generate source code in 17 programming languages, as well as generate documentation from a schema. Furthermore, it is open source, free, accessible as a web service and we are familiar with it.

There also exist approaches to generate a schema automatically, based on an instance dataset. They are referred to as *schema inference* [13] or *schema discovery* [14] tasks. LLM-based services, such as ChatGPT, can also directly be used to generate a schema using natural language, however these services lack proper schema validation and visualization of the schema in a graphical way. Furthermore, 1) LLMs can be distracted by irrelevant contexts [15]; 2) Even for deterministic tasks, LLMs showed a significant drop in accuracy when dealing with low-probability inputs [16]; and 3) The reasoning abilities of LLMs degrades as the input length increases, also before reaching maximum context window [17]. Mior [18] fine-tune a LLM for schema related tasks and outperform the base model Code LLama [19] significantly.

C. Schema Matching and Mapping

Integrating data from various sources and in different formats poses a significant challenge [20]–[22]. In this work, we study the task of schema mapping: converting an instance from one JSON schema to another. A schema mapping can contain simple property-to-property correspondences or also more expressive logics. JSON to JSON transformation (Jolt)[3], is a Java library to transform JSON documents. JSONata[4], is another transformation library, written in TypeScript. The JSON query language jq[5] also can be used to transform JSON documents. While it is a powerful tool for concise JSON querying and transformation on the command line, it is not primarily intended for transforming large JSON documents due to its in-memory processing model and limited support for streaming or parallel execution.

Schema mappings can be created manually, but there also exist automated approaches [23]. The task of identifying which elements in one schema correspond to elements in another is called schema matching. Rahm and Bernstein [24] compare different automated schema matching approaches for relational databases. Stanek and Killough [25] created a program which generates code to convert a JSON document from one schema to another, defining and implementing different mapping rules themselves. Buss et al. [26] discuss the use of LLM to automatically create schema mappings, focusing on relational databases. Among other points, they state that due to the difficult nature of data integration, human-in-the-loop approaches are required.

D. Large Language Models (LLMs) and Prompt Engineering

LLMs are powerful deep learning models trained on large corpora of text to perform a wide range of natural language processing tasks [27]–[29]. Prompt engineering is the task of optimizing the input prompt for an LLM to get the best results from it. Sahoo et al. [30] performed a systematic survey of prompt engineering techniques. For example, for new tasks without extensive prior training, few-shot training (providing the model a few input-output examples) improves model performance on complex tasks, but requires additional tokens and can lead to biases. Assigning the LLM a role or persona can also increase the model performance [31]. For complex long instructional prompts, reframing them into multiple smaller subtasks has shown to improve the results [32]. Prior work has shown that providing detailed natural-language instructions significantly improves task performance, compared to relying on implicit task formulation [33]–[35].

III. Design and Implementation

In this section, we describe the design and implementation of our AI-assisted schema creation (Section III-A) and schema matching (Section III-B) approaches.

Our system communicates with LLMs via a configurable endpoint that follows the OpenAI API[6]. Users may select the desired model via the application's settings. Prompts are programmatically constructed and transmitted to the API, and the resulting textual outputs are automatically parsed and processed by our application. Access to LLMs via these APIs requires an authentication key tied to a user account and billing configuration. We do not provide such API keys directly; instead, MetaConfigurator requires users to supply their own credentials for API access. All LLM-based interactions and evaluations presented in this paper were conducted using

[2]https://www.liquid-technologies.com/json-schema-editor, acc. 25/06/02
[3]https://github.com/bazaarvoice/jolt, acc. 25/06/02
[4]https://github.com/jsonata-js/jsonata, acc. 25/06/02
[5]https://github.com/jqlang/jq, acc. 25/06/02
[6]https://platform.openai.com/docs, acc. 25/06/07

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on March 10,2026 at 14:36:17 UTC from IEEE Xplore. Restrictions apply.

`gpt-4o-mini`, which we selected for its favorable trade-off between cost-efficiency and its demonstrated proficiency in handling JSON Schema tasks.

A. AI-assisted JSON Schema Creation

MetaConfigurator's modular and extensible architecture enables the integration of a new *AI assistance* view. To enable schema creation and modifications via natural language, we introduce a conversational, chat-like interface. Our hybrid method offers several advantages over pure LLM-based solutions:

1) **Prompt construction and context management**: The tool handles all aspects of prompt engineering. For schema creation, it dynamically constructs a structured prompt that sets the LLM's role (e.g., *"You are a JSON Schema expert"*), includes the user's natural language description, and specifies the expected output format. For schema modifications, the relevant schema subset, based on the user's current selection, is included in the prompt.
2) **Integrated validation and visualization**: The generated or modified schema is immediately subject to validation and is visualized within MetaConfigurator's schema editor.
3) **Scalability through targeted context**: To prevent LLM inaccuracies and hallucinations on large schemas, we avoid transmitting the entire schema. Instead, only the user-selected sub-schema is provided as context, enabling precise and modular editing.
4) **Automated response post-processing**: The tool automatically cleans the LLM response by removing formatting artifacts such as code fences or language identifiers.
5) **Human-in-the-loop editing**: If the LLM output is incomplete or incorrect, the user is presented with the raw response. They may then correct the schema manually before accepting or discarding the change.

For schema creation tasks, prompts are constructed according to the principles described above. Figure 1 shows an example of a user-provided schema description to MetaConfigurator and the resulting schema, visualized. The same chat-based interface can be used to modify existing schemas (Figure 2), to query a schema for information or to create, edit and query document instances.

B. AI-assisted Schema Matching and Mapping

Schema mapping is performed using human-defined mapping rules or scripts, traditional automated approaches [23], [25] and recently also using LLMs [26]. We propose and implement a new approach, which transforms a JSON document to satisfy a given target schema by first generating schema mapping rules with the help of a LLM and then executing these deterministically. Aside from JSON, MetaConfigurator supports the data formats YAML and XML and has a function to import CSV documents (by conversion to JSON). Therefore, data from any of these formats are supported and can be transformed using the approach.

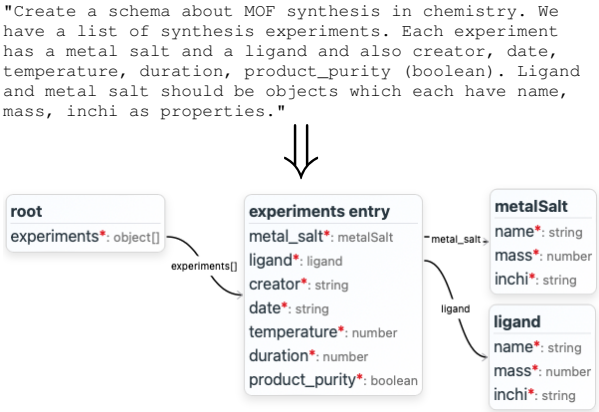

Fig. 1: Example of natural language schema creation: The user provides a prompt (top), which is translated into a structured visual schema (bottom).

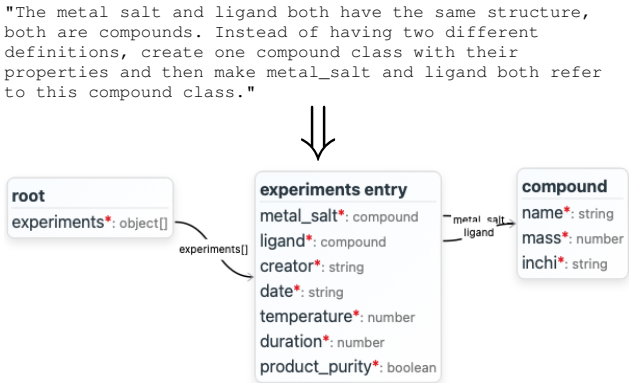

Fig. 2: Example of schema modification through natural language: The user requests schema changes using a prompt (top), which results in the updated schema (bottom).

As schema mapping language, we use JSONata. It is very expressive and supports complex features, such as numeric, path, boolean and comparison operators, sorting, grouping, aggregation, functions and expressions, functional programming and higher order functions.

To perform the schema matching and mapping (data model transformation), the user provides an input JSON document and a target JSON schema. First, a source schema is inferred for the provided JSON document instance, using the jsonhero/schema-infer[7] library. Next, the prompt for the LLM is constructed. It consists of mapping instructions (giving the LLM a personality, describing the overall tasks and detailed mapping rules), a mapping example (example input and expected output files) and the user input. To improve the quality of the generated JSONata expressions, a detailed set of instructions and JSONata specifications[8] is included.

[7]https://github.com/triggerdotdev/schema-infer, acc. 25/06/05.
[8]Our set of JSONata instructions: https://doi.org/10.18419/DARUS-5157

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on March 10,2026 at 14:36:17 UTC from IEEE Xplore. Restrictions apply.

Because the user input JSON document might be large, we perform a recursive array and properties truncation to shorten the document while maintaining its overall structure. The algorithm iteratively trims all arrays and object properties to a maximum length n and $8 * n$ respectively, until either the target document size of `64KB` or the minimum $n_{min} = 2$ is reached. The value `n` starts at `64` and every iteration it is divided by 2. Properties of objects are trimmed more conservatively than array items, to a length of $8 * n$, as they are usually all relevant and different (except for schemas with `additionalProperties` or `patternProperties`), contrary to arrays items, which tend to follow the same schema. Possible information loss is constrained by including the JSON schema inferred from the complete JSON document in the prompt. Because of including the whole document schema in the prompt, there is an upper boundary of document complexity which is supported.

After the prompt is constructed, sent to the LLM and a response is received, a post-processing algorithm is executed on the response. The response might be surrounded by a code fence, often with a language hint (`jsonata`). If applicable, it is removed. Finally, the resulting suggested schema mapping is presented to the user in an interactive code editor. It is automatically validated for syntactical correctness (not semantics) using the JSONata library[9] and validation errors are shown to the user. The user can make any changes of their choice and can only apply the mapping once it is recognized as valid. Figure 3 shows an example schema mapping input, the generated JSONata expression and the transformation result.

Input JSON Document

```
{
  "mof_id": "mof-123",
  "reagents": [
    { "role": "metal", "name": "ZrCl4" },
    { "role": "linker", "name": "BDC" },
    { "role": "linker", "name": "DABCO" }
  ]
}
```

Target Schema

```
{
  "type": "object",
  "properties": {
    "materialId": { "type": "string" },
    "metal": { "type": "string" },
    "linkers": {
      "type": "array",
      "items": { "type": "string" }
    }
  }
}
```

$\Downarrow$

Mapping Generated by LLM

```
{
  "materialId": mof_id,
  "metal": reagents[role = "metal"].name,
  "linkers": reagents[role = "linker"].name
}
```

Transformed JSON Document

```
{
  "materialId": "mof-123",
  "metal": "ZrCl4",
  "linkers": ["BDC", "DABCO"]
}
```

Fig. 3: Example of schema mapping: given an input JSON document and a target schema (top), the LLM generates a transformation expression (middle), which is then used to derive the structured output (bottom).

IV. Application Example

Chemists have performed chemical synthesis experiments and tracked their inputs and results in an Excel table. They want to apply machine learning and other algorithms on their data, but it is not yet in a suitable format[10]. MetaConfigurator is used to convert the excel table into a JSON document and automatically infer the corresponding schema. The JSON document and schema, however, are not yet of sufficient quality. Among other points, the `product_purity` values of `yes` and `no` have been inferred as being of type `string` instead of `boolean` and the `ligand` and `metal_salt` properties are all listed directly as children of an experiment instance, although it would be more suitable to extract them into separate `ligand` and `metal_salt` classes. Using the chat-based interface we create an improved schema (Figure 1-2). To transform the JSON document to satisfy the new schema, we use the automated schema mapping functionality. It successfully maps from `yes` and `no` to boolean values and from the flat source paths to the nested paths of the target schema. The data is now well-structured, formally specified by a schema and AI-ready.

For the automated execution of experiments using laboratory robots, existing standards are preferred. For example, the creators of Chemspyd [36], a python interface for operating laboratory robotic platforms from Chemspeed Technologies, provide the outlook of integrating the Chemical Description Language XDL [37] into Chemspyd. As of writing this paper, no formal schema is available for XDL 2.0, aside from documentation[11]. We copy and paste the relevant sections of the XDL documentation into the schema creation prompt of MetaConfigurator to generate a schema. Since XDL documents fundamentally differ from our data in structure, JSONata cannot provide an adequate mapping. Therefore, we use the code generation capabilities of MetaConfigurator to generate python classes representing the data structure of our schema and the XDL schema and write our own python code to convert between the structures. The result is a XDL document instance that is compatible with the XDL ecosystem and might enable an automatic synthesis execution in the near future.

V. Conclusion

We presented a hybrid approach for schema creation, editing, and instance transformation using large language models, integrated into the open-source tool MetaConfigurator. Our method combines modular prompt engineering, targeted context scoping, human-in-the-loop refinement, and post-processing safeguards to enable intuitive and controlled schema modeling. To enable transforming also large document instances to satisfy a target schema, mapping rules are generated by LLMs but then applied in a deterministic manner.

An example in the chemistry domain demonstrates the applicability of our approach. By translating informal data

[9] https://github.com/jsonata-js/jsonata, acc. 25/06/09.

[10] Sources: https://doi.org/10.18419/DARUS-5157.

[11] https://croningroup.gitlab.io/chemputer/xdl/standard, acc. 25/07/05.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on March 10,2026 at 14:36:17 UTC from IEEE Xplore. Restrictions apply.

into machine-readable structures, our method enables FAIR (Findable, Accessible, Interoperable, and Reusable) [38] data practices and model-driven workflows.

Beyond this proof-of-concept, model editing and instance transformation, the tool supports other MDE-oriented tasks such as instance validation, code/documentation generation, mapping discovery and schema-driven form generation. A future extension for model-to-model transformations (e.g., XSD to JSON Schema) is planned.

Acknowledgment

The authors acknowledge Esengül Ciftci (Max Planck Institute for Solid State Research, Stuttgart, Germany) and Kenichi Endo (University of Stuttgart, Institute of Polymer Chemistry, Stuttgart, Germany) for inspiring the chemistry application example. The assistance of ChatGPT-4 for editorial suggestions is acknowledged.

References

[1] Douglas C. Schmidt, "Guest editor's introduction: Model-driven engineering," Computer, vol. 39, no. 2, pp. 25–31, 2006, doi: 10.1109/MC.2006.58.

[2] G. Pryor, "Managing research data," Facet Publishing, 2012, doi: 10.29085/9781856048910.

[3] S. Wall, "Data management for researchers: Organize, maintain and share your data for research success," Archives and Manuscripts, vol. 45, no. 1, pp. 53–55, 2017, doi: 10.1080/01576895.2017.1279038.

[4] N. Hartl, E. Wößner, and Y. Sure-Vetter, "Nationale Forschungsdateninfrastruktur (NFDI)," Informatik Spektrum, vol. 44, no. 5, pp. 370–373, 2021.

[5] F. Neubauer, P. Bredl, M. Xu, K. Patel, J. Pleiss, and B. Uekermann, "MetaConfigurator: A user-friendly tool for editing structured data files," Datenbank-Spektrum, pp. 1–9, 2024, doi: 10.1007/s13222-024-00472-7.

[6] D. Crockford, "The application/json media type for JavaScript Object Notation (JSON)," Internet Engineering Task Force IETF Request for Comments, 2006.

[7] O. Ben-Kiki, C. Evans, and I. döt Net, "YAML ain't markup language version 1.2," Available: http://yaml.org/spec/1.2/spec.html, 2009.

[8] F. Yergeau, T. Bray, J. Paoli, C. M. Sperberg-McQueen, and E. Maler, "Extensible markup language (XML) 1.0," in XML, vol. 1, no. 1, pp. W3C, 2006.

[9] T. Marrs, "JSON at work: Practical data integration for the web," O'Reilly Media, Inc., 2017, pp. 269–286.

[10] M.-A. Baazizi, D. Colazzo, G. Ghelli, C. Sartiani, and S. Scherzinger, "An empirical study on the "Usage of Not" in real-world JSON schema documents (long version)," arXiv preprint arXiv:2107.08677, 2021, unpublished.

[11] I. C. Siffa, J. Schäfer, and M. M. Becker, "Adamant: A JSON schema-based metadata editor for research data management workflows," F1000Research, vol. 11, p. 475, Apr. 2022, doi: 10.12688/f1000research.110875.1.

[12] F. Neubauer, J. Pleiss, and B. Uekermann, "Data model creation with MetaConfigurator," in Datenbanksysteme für Business, Technologie und Web (BTW 2025), Lecture Notes in Informatics (LNI), vol. P-361, GI, Bonn, 2025, doi: 10.18420/BTW2025-60.

[13] M.-A. Baazizi, D. Colazzo, G. Ghelli, and C. Sartiani, "Parametric schema inference for massive JSON datasets," The VLDB Journal, vol. 28, pp. 497–521, 2019, doi: 10.1007/s00778-018-0532-7.

[14] W. Spoth, O. Kennedy, Y. Lu, B. Hammerschmidt, and Z. H. Liu, "Reducing ambiguity in JSON schema discovery," in Proc. 2021 Int. Conf. on Management of Data, pp. 1732–1744, 2021.

[15] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. H. Chi, N. Schärli, and D. Zhou, "Large language models can be easily distracted by irrelevant context," in Int. Conf. on Machine Learning, pp. 31210–31227, 2023.

[16] R. T. McCoy, S. Yao, D. Friedman, M. Hardy, and T. L. Griffiths, "Embers of autoregression: Understanding large language models through the problem they are trained to solve," arXiv preprint arXiv:2309.13638, 2023, unpublished.

[17] M. Levy, A. Jacoby, and Y. Goldberg, "Same task, more tokens: The impact of input length on the reasoning performance of large language models," arXiv preprint arXiv:2402.14848, 2024, unpublished.

[18] M. J. Mior, "Large language models for JSON schema discovery," arXiv preprint arXiv:2407.03286, 2024, unpublished.

[19] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, et al., "Code LLaMA: Open foundation models for code," arXiv preprint arXiv:2308.12950, 2023, unpublished.

[20] I. M. Putrama and P. Martinek, "Heterogeneous data integration: Challenges and opportunities," Data in Brief, p. 110853, 2024.

[21] B. Ahamed and T. Ramkumar, "Data integration—Challenges, techniques and future directions: A comprehensive study," Indian Journal of Science and Technology, vol. 9, no. 44, pp. 1–9, 2016.

[22] F. Z. Rozony, M. N. A. Aktar, M. Ashrafuzzaman, and A. Islam, "A systematic review of big data integration challenges and solutions for heterogeneous data sources," Academic Journal on Business Administration, Innovation & Sustainability, vol. 4, no. 4, pp. 1–18, 2024.

[23] R. Hai, C. Quix, and D. Kensche, "Nested schema mappings for integrating JSON," in Conceptual Modeling: Proc. ER 2018, Xi'an, China, Springer, pp. 397–405, 2018.

[24] E. Rahm and P. A. Bernstein, "A survey of approaches to automatic schema matching," The VLDB Journal, 10:334–350, 2001. Springer, doi: 10.1007/s007780100057.

[25] J. Stanek and D. Killough, "Synthesizing JSON Schema Transformers," arXiv preprint arXiv:2405.17681, 2024, unpublished.

[26] C. Buss, M. Safari, A. Termehchy, S. Lee, and D. Maier, "Towards Scalable Schema Mapping using Large Language Models," arXiv preprint arXiv:2505.24716, 2025, unpublished.

[27] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., "Language models are few-shot learners," Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901, 2020, doi: 10.48550/arXiv.2005.14165

[28] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al., "GPT-4 technical report," arXiv preprint arXiv:2303.08774, 2023, unpublished.

[29] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., "LLaMA: Open and efficient foundation language models," arXiv preprint arXiv:2302.13971, 2023, unpublished.

[30] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications," arXiv preprint arXiv:2402.07927, 2024, unpublished.

[31] A. Kong, S. Zhao, H. Chen, Q. Li, Y. Qin, R. Sun, X. Zhou, E. Wang, and X. Dong, "Better zero-shot reasoning with role-play prompting," arXiv preprint arXiv:2308.07702, 2023.

[32] S. Mishra, D. Khashabi, C. Baral, Y. Choi, and H. Hajishirzi, "Reframing instructional prompts to GPTk's language," arXiv preprint arXiv:2109.07830, 2021, unpublished.

[33] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al., "Training language models to follow instructions with human feedback," Advances in Neural Information Processing Systems, vol. 35, pp. 27730–27744, 2022.

[34] Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi, and H. Hajishirzi, "Self-instruct: Aligning language models with self-generated instructions," arXiv preprint arXiv:2212.10560, 2022, unpublished.

[35] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," arXiv preprint arXiv:2109.01652, 2021, unpublished.

[36] M. Seifrid, F. Strieth-Kalthoff, M. Haddadnia, T. C. Wu, E. Alca, L. Bodo, S. Arellano-Rubach, N. Yoshikawa, M. Skreta, and R. Keunen, "Chemspyd: An open-source Python interface for Chemspeed robotic chemistry and materials platforms," Digital Discovery, vol. 3, no. 7, pp. 1319–1326, 2024.

[37] S. H. M. Mehr, M. Craven, A. I. Leonov, G. Keenan, and L. Cronin, "A universal system for digitization and automatic execution of the chemical synthesis literature," Science, vol. 370, no. 6512, pp. 101–108, 2020, doi: 10.1126/science.abc2986.

[38] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, et al., "The FAIR guiding principles for scientific data management and stewardship," Scientific Data, vol. 3, no. 1, pp. 1–9, 2016.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on March 10,2026 at 14:36:17 UTC from IEEE Xplore. Restrictions apply.